

Chapter 46

I2S Controller (I2S)

46.1 Overview

ESP32-P4 has three built-in I2S interfaces (i.e., I2S0, I2S1, and I2S2), which provide flexible communication interfaces for streaming digital data in multimedia applications, especially digital audio applications.

The I2S standard bus defines three signals, namely, a bit clock signal (BCK), a channel/word select signal (WS), and a serial data signal (SD). A basic I2S data bus has one master and one slave. The roles remain unchanged throughout the communication. The I2S module on ESP32-P4 provides separate transmit (TX) and receive (RX) units for high performance.

Note:

The information provided in this chapter applies to I2S0, I2S1, and I2S2. Unless otherwise indicated, I2S or I2S n in this chapter refer to I2S0, I2S1, and I2S2.

46.2 Terminology

To better illustrate the functionality of I2S, the following terms are used in this chapter.

Master mode	As a master, I2S n drives BCK/WS signals and transmits data to or receives data from a slave.
Slave mode	As a slave, I2S n is driven by BCK/WS signals and receives data from or transmits data to a master.
Full-duplex	There are two separate data lines. The transmitted and received data is carried simultaneously.
Half-duplex	Only one side, the master or the slave, transmits data first, and the other side receives data. Data transmission and reception cannot occur simultaneously.
A-law and μ-law	A-law and μ -law are compression/decompression algorithms in digital pulse code modulated (PCM) non-uniform quantization, which can effectively improve the signal-to-quantization noise ratio.
TDM RX mode	In this mode, pulse code modulated (PCM) data is received utilizing time division multiplexing (TDM). The signal lines include BCK, WS, and SD. Data from 16 channels at most can be received. TDM Philips standard, TDM MSB alignment standard, and TDM PCM standard are supported in this mode, depending on user configuration.

PDM RX mode	In this mode, pulse density modulation (PDM) data is received. Used signals include WS and DATA. PDM standard is supported in this mode by user configuration.
TDM TX mode	In this mode, pulse code modulated (PCM) data is sent in a way of time division multiplexing (TDM). The signal lines include BCK, WS, and DATA. Data up to 16 channels can be sent. TDM Philips standard, TDM MSB alignment standard, and TDM PCM standard are supported in this mode, depending on user configuration.
PDM TX mode	In this mode, pulse density modulation (PDM) data is sent. The signal lines include WS and DATA. PDM standard is supported in this mode by user configuration.
PCM-to-PDM Data	
Format Converter (for I2S only)	A data format converter that converts PCM data into PDM data (hereinafter referred to as PCM-to-PDM converter), which can be enabled in PDM TX master mode. When the converter is enabled, I2S fetches pulse code modulated (PCM) data from storage via GDMA, converts it to pulse density modulation (PDM) data, and then transmits it.
PDM-to-PCM Data	
Format Converter (for I2S only)	A data format converter that converts PDM data into PCM data (hereinafter referred to as PDM-to-PCM converter), which can be enabled in PDM RX master or slave mode. When the converter is enabled, I2S receives pulse density modulation (PDM) data, converts it into pulse code modulated (PCM) data, and stores it into memory via GDMA.

46.3 Features

The I2S module has the following features:

- Master mode and slave mode
- Full-duplex and half-duplex communications
- Separate TX and RX units that can work independently or simultaneously
- A variety of audio standards supported:
 - TDM Philips standard
 - TDM MSB alignment standard
 - TDM PCM standard
 - PDM standard
- Various TX/RX modes supported:
 - TDM TX mode, up to 16 channels supported
 - TDM RX mode, up to 16 channels supported

- PDM TX mode
 - * Raw PDM data transmission
 - * PCM-to-PDM data format conversion (for I2S₀ only), up to 2 channels supported
- PDM RX mode
 - * Raw PDM data reception
 - * PDM-to-PCM data format conversion (for I2S₀ only), up to 8 channels supported
- Configurable APLL clock with frequencies up to 240 MHz
- Configurable high-precision sample clock with a variety of sampling frequencies supported (refer to the note below for more details)
- 8/16/24/32-bit data width
- Synchronous counter in TX mode
- ETM feature
- Direct Memory Access (GDMA-AHB only)
- Standard I2S interface interrupts

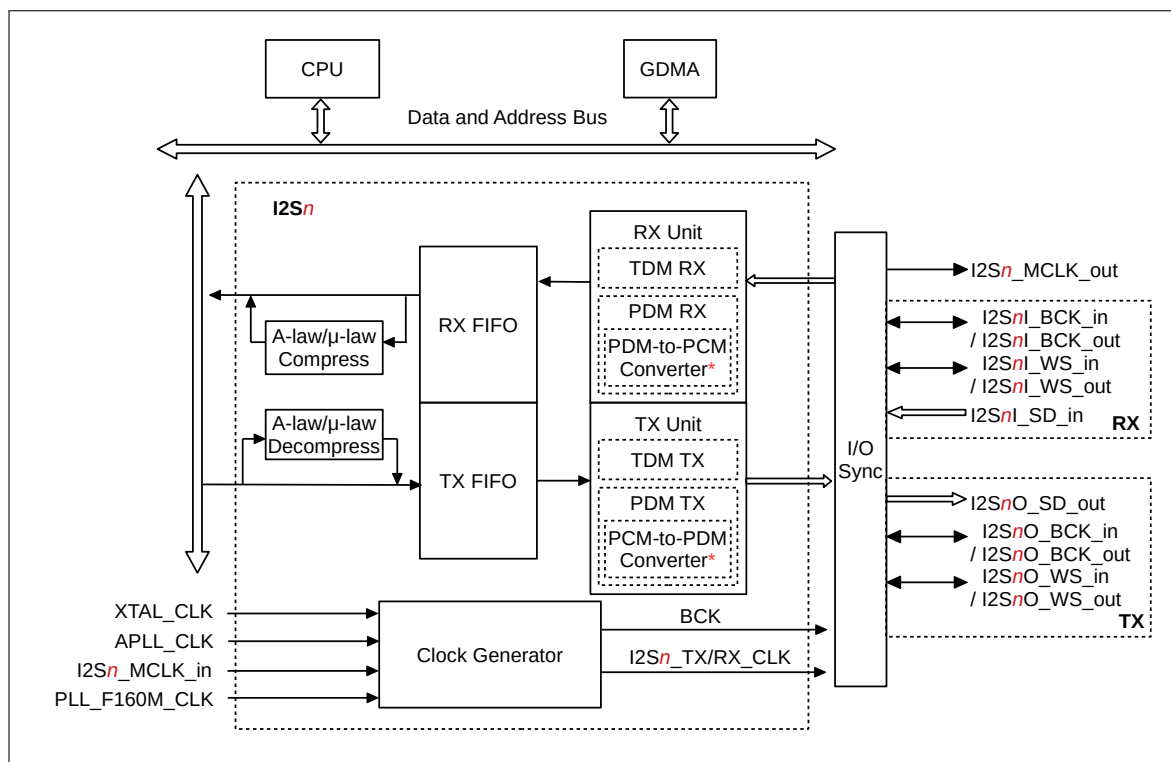
Note:

In slave mode, to ensure the sampling accuracy, the module clock frequency must be greater than or equal to 8 times the BCK clock frequency. Therefore, the maximum sampling frequency of I2S_n is limited by the data bit width and the number of channels. For example, since the clock source frequency is up to 240 MHz, the module clock can be configured up to 120 MHz, and the BCK clock can be configured up to 15 MHz. Therefore, when transmitting dual-channel 32-bit width data, I2S_n supports sample frequencies up to 234.375 kHz, e.g., 8 kHz, 16 kHz, 32 kHz, 44.1 kHz, 48 kHz, 88.2 kHz, 96 kHz, 128 kHz, and 192 kHz. Please refer to Section [46.6 I2S TX/RX Clock](#) for detailed information.

46.4 System Architecture

Figure [46.4-1](#) shows the structure of the ESP32-P4 I2S_n module, consisting of:

- Transmit control unit (TX Unit)
- Receive control unit (RX Unit)
- Input and output timing unit (I/O Sync)
- Clock divider (Clock Generator)
- 64 x 32-bit TX FIFO
- 64 x 32-bit RX FIFO
- Compress/Decompress units



Note: The PDM-to-PCM converter and PCM-to-PDM converter are only supported by I2S0.

Figure 46.4-1. ESP32-P4 I2S System Diagram

The I2S_n module supports direct memory access (DMA) to internal memory. For more information, see Chapter 4 *GDMA Controller (GDMA-AHB, GDMA-AXI)*.

Both the TX unit and the RX unit have a three-line interface that uses a bit clock line (BCK), a word select line (WS), and a serial data line (SD). The SD line of the TX unit is used for data output and the SD line of the RX unit for data input. The BCK and WS signal lines of the TX and RX units are used for data output in master mode, and the BCK and WS signal lines of the TX and RX units are used for data input in slave mode.

The signal bus of the I2S_n module is shown at the right part of Figure 46.4-1. The naming of these signals in RX and TX units follows the pattern of I2S_nA_B_C such as I2S_nI_BCK_in.

- “A” indicates that the signal belongs to the I2S_n TX or RX unit, which includes:
 - “I”: Signal input to/output from the RX unit
 - “O”: Signal input to/output from the TX unit
- “B” represents the signal function, which includes:
 - BCK
 - WS
 - SD
- “C” represents the signal direction, which includes:
 - “in”: Input signal into the I2S_n module
 - “out”: Output signal from the I2S_n module

Table 46.4-1 provides a detailed description of I2S signals.

Table 46.4-1. I2S_n Signal Description

Signal	Direction	Function
I2S _n I_BCK_in	Input	In I2S _n slave mode, inputs BCK signal for RX unit.
I2S _n I_BCK_out	Output	In I2S _n master mode, outputs BCK signal for RX unit.
I2S _n I_WS_in	Input	In I2S _n slave mode, inputs WS signal for RX unit.
I2S _n I_WS_out	Output	In I2S _n master mode, outputs WS signal for RX unit.
I2S _n I_Data_in	Input	Works as the serial input data bus for I2S _n RX unit.
I2S _n O_Data_out	Output	Works as the serial output data bus for I2S _n TX unit.
I2S _n O_BCK_in	Input	In I2S _n slave mode, inputs BCK signal for TX unit.
I2S _n O_BCK_out	Output	In I2S _n master mode, outputs BCK signal for TX unit.
I2S _n O_WS_in	Input	In I2S _n slave mode, inputs WS signal for TX unit.
I2S _n O_WS_out	Output	In I2S _n master mode, outputs WS signal for TX unit.
I2S _n _MCLK_in	Input	In I2S _n slave mode, works as a clock source from the external master.
I2S _n _MCLK_out	Output	In I2S _n master mode, works as a clock source for the external slave.
I2S ₀ I_Data1_in	Input	When the PDM-to-PCM converter is enabled, works as the serial input data line for RX unit.
I2S ₀ I_Data2_in	Input	When the PDM-to-PCM converter is enabled, works as the serial input data line for RX unit.
I2S ₀ I_Data3_in	Input	When the PDM-to-PCM converter is enabled, works as the serial input data line for RX unit.
I2S ₀ O_Data1_out	Output	When the PCM-to-PDM converter is enabled, works as the serial output data line for TX unit.

* Any required signals of I2S_n must be mapped to the chip's pins via HP GPIO matrix, see Chapter 9 [GPIO Matrix and IO MUX](#).

46.5 Supported Audio Standards

ESP32-P4 I2S_n supports multiple audio standards, including TDM Philips standard, TDM MSB alignment standard, TDM PCM standard, and PDM standard.

Select the needed standard by configuring the following bits:

- [I2S_TX/RX_TDM_EN](#)
 - 0: Disable TDM mode
 - 1: Enable TDM mode
- [I2S_TX/RX_PDM_EN](#)
 - 0: Disable PDM mode
 - 1: Enable PDM mode
- [I2S_TX/RX_MSB_SHIFT](#)
 - 0: WS and SD signals change simultaneously, i.e., enable MSB alignment standard

- 1: WS signal changes one BCK clock cycle earlier than SD signal, i.e., enable Philips standard or select PCM standard

Note:

`I2S_TX/RX_TDM_EN` and `I2S_TX/RX_PDM_EN` must not be configured to 1 or 0 at the same time, otherwise ESP32-P4 I2S_n will transmit data incorrectly in a mode that is neither TDM nor PDM.

46.5.1 TDM Philips Standard

Philips specifications require that WS signal changes one BCK clock cycle earlier than SD signal on BCK falling edge, which means that WS signal is valid from one clock cycle before transmitting the first bit of channel data and changes one clock before the end of channel data transfer. SD signal line transmits the most significant bit of audio data first.

Compared with the Philips standard, TDM Philips standard supports multiple channels. See Figure 46.5-1.

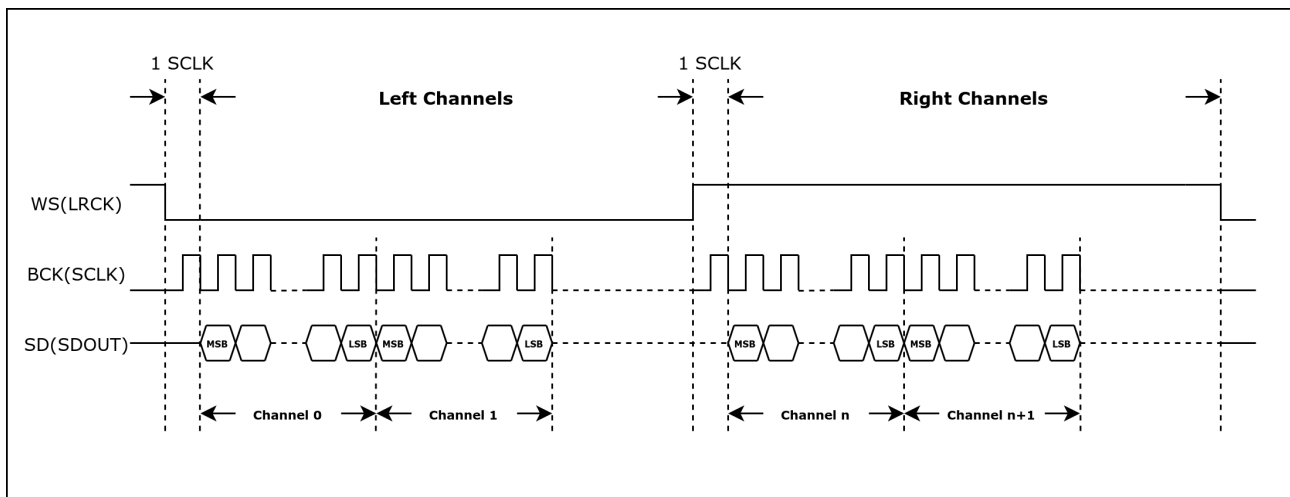


Figure 46.5-1. TDM Philips Standard Timing Diagram

46.5.2 TDM MSB Alignment Standard

MSB alignment specifications require that WS and SD signals change simultaneously on the falling edge of BCK. The WS signal is valid until the end of the channel data transfer. The SD signal line transmits the most significant bit of audio data first.

Compared with MSB alignment standard, TDM MSB alignment standard supports multiple channels. See Figure 46.5-2.

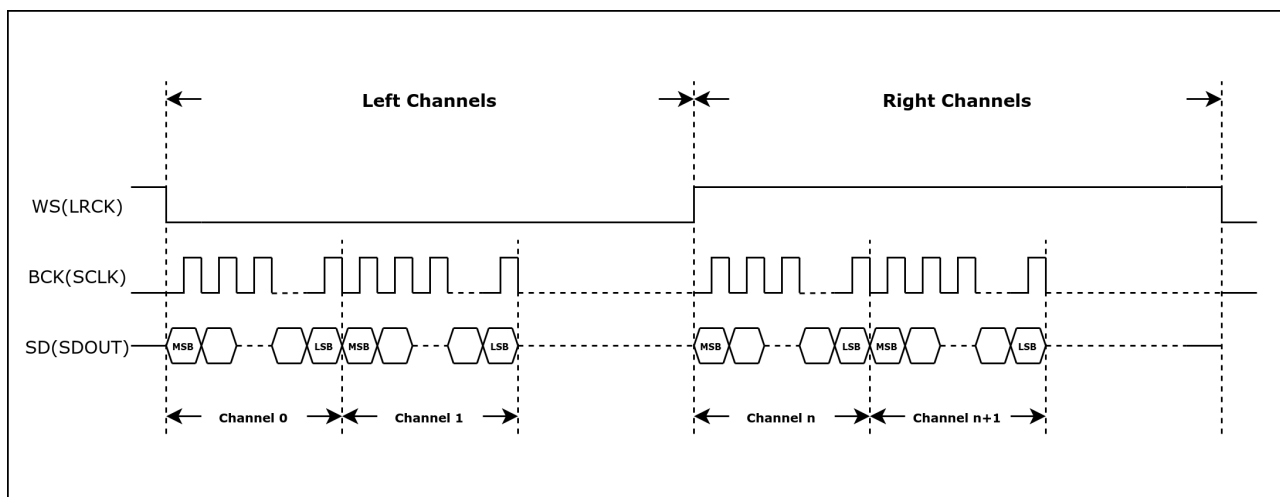


Figure 46.5-2. TDM MSB Alignment Standard Timing Diagram

46.5.3 TDM PCM Standard

Short frame synchronization under the PCM standard requires that the WS signal changes one BCK clock cycle earlier than the SD signal on the falling edge of BCK, which means that the WS signal becomes valid one clock cycle before transferring the first bit of channel data and remains unchanged in this BCK clock cycle. SD signal line first transmits the most significant bit of audio data.

Compared with PCM standard, TDM PCM standard supports multiple channels. See Figure 46.5-3.

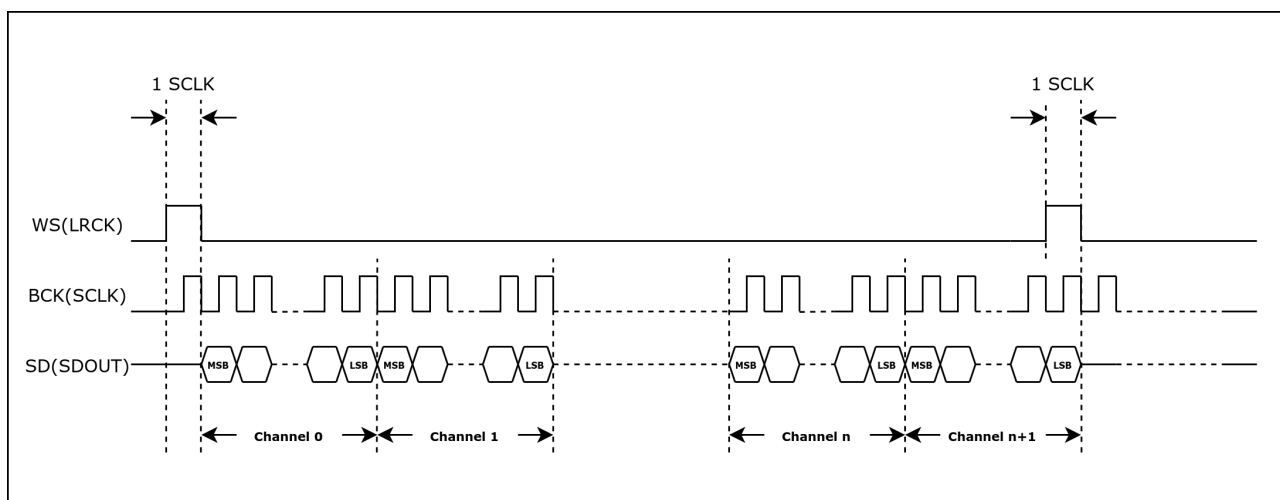


Figure 46.5-3. TDM PCM Standard Timing Diagram

46.5.4 PDM Standard

Under PDM standard, WS signal changes continuously during data transmission. The low-level and high-level of this signal indicates the left channel and right channel respectively. WS and SD signals change simultaneously. See Figure 46.5-4.

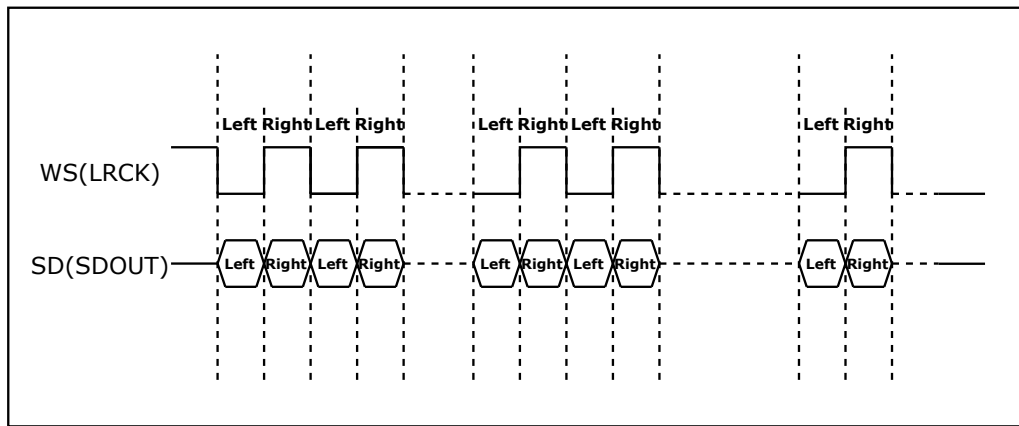


Figure 46.5-4. PDM Standard Timing Diagram

46.6 I2S TX/RX Clock

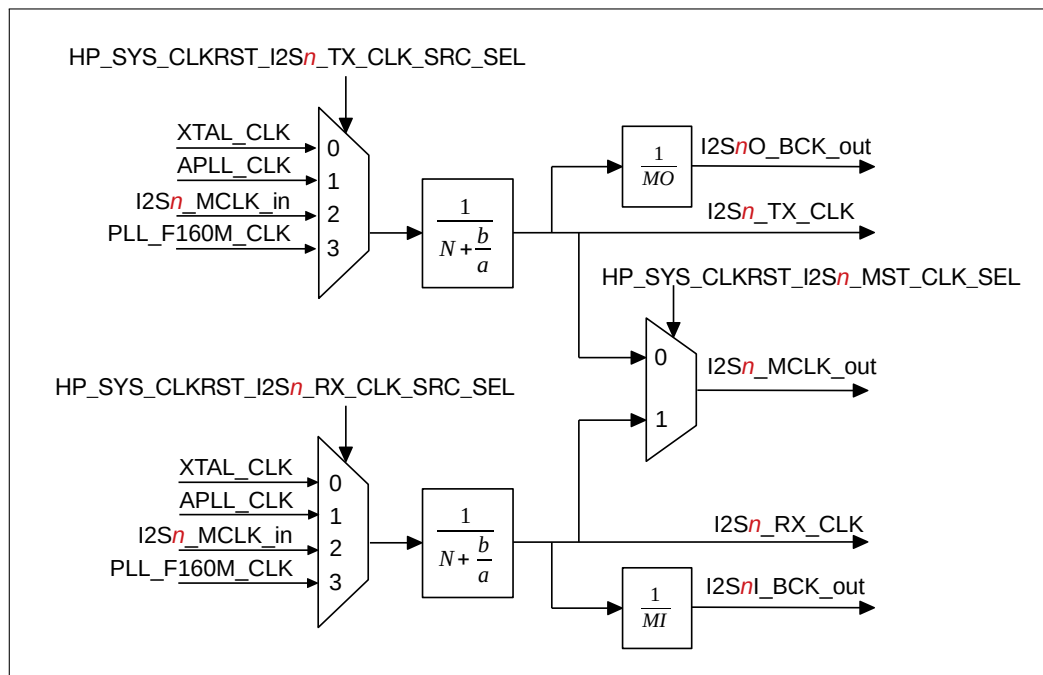
I2S_n_TX/RX_CLK is the master clock of I2S_n TX/RX unit, divided from:

- 40 MHz XTAL_CLK
- 160 MHz PLL_F160M_CLK
- APLL_CLK with adjustable frequency
- I2S_n_MCLK_in (external input clock)

The serial clock (BCK) of the I2S_n TX/RX unit is divided from I2S_n_TX/RX_CLK, as shown in Figure 46.6-1.

HP_SYS_CLKRST_I2S_n_TX/RX_CLK_SRC_SEL is used to select clock source for TX/RX unit, and

HP_SYS_CLKRST_I2S_n_TX/RX_CLK_EN to enable or disable the clock source.

Figure 46.6-1. I2S_n Clock Generator

The following formula shows the relation between I2S_n_TX/RX_CLK frequency $f_{I2S_n_TX/RX_CLK}$ and the divider clock source frequency $f_{I2S_n_CLK_S}$:

$$f_{I2S_n_TX/RX_CLK} = \frac{f_{I2S_n_CLK_S}}{N + \frac{b}{a}}$$

N is an integer value between 2 and 256. The value of N is mapped to that of HP_SYS_CLKRST_I2S_n_TX/RX_DIV_N as follows:

- When HP_SYS_CLKRST_I2S_n_TX/RX_DIV_N = 0, N = 256;
- When HP_SYS_CLKRST_I2S_n_TX/RX_DIV_N = 1, N = 2;
- When HP_SYS_CLKRST_I2S_n_TX/RX_DIV_N has any other value, N = HP_SYS_CLKRST_I2S_n_TX/RX_DIV_N.

The values of “a” and “b” in fractional divider depend only on x, y, z, and yn1. The corresponding formulas are as follows:

- When $b \leq \frac{a}{2}$, $yn1 = 0$, $x = \text{floor}(\lceil \frac{a}{b} \rceil) - 1$, $y = a \% b$, $z = b$;
- When $b > \frac{a}{2}$, $yn1 = 1$, $x = \text{floor}(\lceil \frac{a}{a-b} \rceil) - 1$, $y = a \% (a - b)$, $z = a - b$.

The values of x, y, z, and yn1 are configured in HP_SYS_CLKRST_I2S_n_TX/RX_DIV_X, HP_SYS_CLKRST_I2S_n_TX/RX_DIV_Y, HP_SYS_CLKRST_I2S_n_TX/RX_DIV_Z, and HP_SYS_CLKRST_I2S_n_TX/RX_DIV_YN1.

To configure the integer divider, clear HP_SYS_CLKRST_I2S_n_TX/RX_DIV_X and HP_SYS_CLKRST_I2S_n_TX/RX_DIV_Z, then set HP_SYS_CLKRST_I2S_n_TX/RX_DIV_Y to 1.

Note:

Using fractional divider may introduce some clock jitter.

In master TX mode, the serial clock BCK for I2S_n TX unit is I2S_nO_BCK_out divided from I2S_n_TX_CLK which is:

$$f_{I2S_nO_BCK_out} = \frac{f_{I2S_n_TX_CLK}}{MO}$$

“MO” is an integer value:

$$MO = I2S_n_TX_BCK_DIV_NUM + 1$$

Note:

Note that I2S_TX_BCK_DIV_NUM must not be configured as 1.

In master RX mode, the serial clock BCK for I2S_n RX unit is I2S_nI_BCK_out divided from I2S_n_RX_CLK, which is:

$$f_{I2S_nI_BCK_out} = \frac{f_{I2S_n_RX_CLK}}{MI}$$

“MI” is an integer value:

$$MI = I2S_n_RX_BCK_DIV_NUM + 1$$

Note:

- `I2S_RX_BCK_DIV_NUM` must not be configured as 1.
- In I2S slave mode, make sure $f_{I2S_TX/RX_CLK} \geq 8 \times f_{BCK}$. The I2S_n module can output I2S_n_MCLK_out as the master clock for peripherals.

46.7 I2S Reset

The units and FIFOs in the I2S_n module are reset by the following bits.

- I2S_n TX/RX units: Reset by the bits `I2S_TX_RESET` and `I2S_RX_RESET`;
- I2S_n TX/RX FIFO: Reset by the bits `I2S_TX_FIFO_RESET` and `I2S_RX_FIFO_RESET`.

Note:

The I2S_n module clock must be configured first before the module and FIFO are reset.

46.8 I2S Master/Slave Mode

The ESP32-P4 I2S_n module can operate as a master or a slave in half-duplex and full-duplex communications, depending on the configuration of `I2S_RX_SLAVE_MOD` and `I2S_TX_SLAVE_MOD`.

- `I2S_TX_SLAVE_MOD`
 - 0: Master TX mode
 - 1: Slave TX mode
- `I2S_RX_SLAVE_MOD`
 - 0: Master RX mode
 - 1: Slave RX mode

46.8.1 Master/Slave TX Mode

- I2S_n works as a master transmitter:
 - Set `I2S_TX_START` to start transmitting data. When this bit is set, the TX unit keeps driving the clock signal and serial data.
 - If `I2S_TX_STOP_EN` is set and all the data in FIFO is transmitted, the master stops transmitting data and clock signals.
 - If `I2S_TX_STOP_EN` is cleared and all the data in FIFO is transmitted, meanwhile no new data is filled into FIFO, the TX unit keeps transmitting the last data frame and clock signal.
 - Master stops transmitting data when the bit `I2S_TX_START` is cleared.
- I2S_n works as a slave transmitter:
 - Set `I2S_TX_START`. Wait for the master BCK clock to enable a transmit operation.

- If [I2S_TX_STOP_EN](#) is set and all the data in FIFO is transmitted, then the slave keeps transmitting zeros, till the master stops providing BCK signal.
- If [I2S_TX_STOP_EN](#) is cleared and all the data in FIFO is transmitted, meanwhile no new data is filled into FIFO, the TX unit keeps transmitting the last data frame.
- If [I2S_TX_START](#) is cleared, slave keeps transmitting zeros till the master stops providing BCK clock signal.

46.8.2 Master/Slave RX Mode

- I2Sn works as a master receiver:
 - Set [I2S_RX_START](#) to start receiving data. When this bit is set, the RX unit keeps outputting clock signal and sampling input data.
 - Configure [I2S_RX_STOP_MODE](#) to control the suspension of data reception:
 - * 0: The RX unit only suspends data reception when [I2S_RX_START](#) is cleared.
 - * 1: The RX unit suspends data reception when [I2S_RX_START](#) is cleared or the number of received bytes is greater than the value configured in [I2S_RX_EOF_NUM_REG](#). When [I2S_RX_START](#) is not cleared, data reception can be restarted by setting [I2S_RX_RESET](#) to 1.
 - * 2: The RX unit suspends data reception when [I2S_RX_START](#) is cleared or GDMA RX FIFO is full. When [I2S_RX_START](#) is not cleared and the GDMA RX FIFO is no longer full, data reception can be restarted by setting [I2S_RX_RESET](#) to 1.
 - RX unit stops receiving data when the bit [I2S_RX_START](#) is cleared.
- I2Sn works as a slave receiver:
 - Set [I2S_RX_START](#). Wait for the master BCK signal to start receiving data.
 - Configure [I2S_RX_STOP_MODE](#) to control data reception suspension:
 - * 0: The RX unit only suspends data reception when [I2S_RX_START](#) is cleared.
 - * 1: The RX unit suspends data reception when the number of received bytes is greater than the value configured in [I2S_RX_EOF_NUM_REG](#). When [I2S_RX_START](#) is not cleared, data reception can be restarted by setting [I2S_RX_RESET](#) to 1.
 - * 2: The RX unit suspends data reception when [I2S_RX_START](#) is cleared or GDMA RX FIFO is full. When [I2S_RX_START](#) is not cleared and the GDMA RX FIFO is no longer full, data reception can be restarted by setting [I2S_RX_RESET](#) to 1.
 - The RX unit stops receiving data when the bit [I2S_RX_START](#) is cleared.

46.9 Transmitting Data

Note:

Updating the configuration described in this and subsequent sections requires to set [I2S_TX_UPDATE](#) accordingly to synchronize registers from APB clock domain to TX clock domain. For more detailed configurations, see Section [46.13.1](#).

In TX mode, I2S_n first reads data through GDMA and transmits these data out via output signals according to the configured data mode and channel mode.

46.9.1 Data Format Control

Data format is controlled in the following phases:

- Phase I: Read data from memory and write it to TX FIFO;
- Phase II: Read the TX data from TX FIFO and convert the data according to the output data mode;
- Phase III: Clock out the TX data serially.

46.9.1.1 Bit Width Control of Channel Valid Data

The bit width of the valid data in each channel is determined by [I2S_TX_BITS_MOD](#) and [I2S_TX_24_FILL_EN](#). For details, see the table below.

Table 46.9-1. Bit Width of Channel Valid Data

Channel Valid Data Width	I2S_TX_BITS_MOD	I2S_TX_24_FILL_EN
32	31	x [*]
	23	1
24	23	0
16	15	x
8	7	x

* “x” represents that this value is ignored.

46.9.1.2 Endian Control of Channel Valid Data

When I2S_n reads data through GDMA, the data endian under various data width is controlled by [I2S_TX_BIG_ENDIAN](#). Table 46.9-2 shows how [I2S_TX_BIG_ENDIAN](#) controls the reading of the data with different valid data widths of the channel.

Table 46.9-2. Endian of Channel Valid Data

Channel Valid Data Width	Original Data	Endian of Processed Data	I2S_TX_BIG_ENDIAN
32	{B3, B2, B1, B0}	{B3, B2, B1, B0}	0
		{B0, B1, B2, B3}	1
24	{B2, B1, B0}	{B2, B1, B0}	0
		{B0, B1, B2}	1
16	{B1, B0}	{B1, B0}	0
		{B0, B1}	1
8	{B0}	{B0}	x

Note:

B0, B1, B2, B3 each represents an 8-bit data, and the symbol {} indicates that the bytes are combined together. For

example, {B3, B2, B1, B0} represents a 32-bit data, wherein B0 represents bit 0-7, B1 represents bit 8-15, B2 represents bit 16-23, and B3 represents bit 24-31.

46.9.1.3 A-law/ μ -law Compression and Decompression

ESP32-P4 I2Sⁿ compresses/decompresses the valid data into 32-bit by A-law or by μ -law. If the bit width of valid data is smaller than 32, zeros are filled to the extra high bits of the data to be compressed/decompressed by default.

Note:

Extra high bits here mean the bits[31: channel valid data width] of the data to be compressed/decompressed.

Configure [I2S_TX_PCM_BYPASS](#):

- 0: Compress or decompress the data
- 1: Do not compress or decompress the data

Configure [I2S_TX_PCM_CONF](#):

- 0: Decompress the data using A-law
- 1: Compress the data using A-law
- 2: Decompress the data using μ -law
- 3: Compress the data using μ -law

At this point, the first phase of data format control is completed.

46.9.1.4 Bit Width Control of Channel TX Data

The TX data width in each channel is determined by [I2S_TX_TDM_CHAN_BITS](#).

- If TX data width in each channel is larger than the valid data width, zeros will be filled to these extra bits.
Configure [I2S_TX_LEFT_ALIGN](#):

- 0: The valid data is at the lower bits of TX data. Zeros are filled into higher bits of TX data;
- 1: The valid data is at the higher bits of TX data. Zeros are filled into lower bits of TX data.

- If the TX data width in each channel is smaller than the valid data width, only the lower bits of valid data are sent out, and the higher bits are discarded.

At this point, the second phase of data format control is completed.

46.9.1.5 Bit Order Control of Channel Data

The data bit order in each channel is controlled by [I2S_TX_BIT_ORDER](#):

- 0: Not reverse the valid data bit order;
- 1: Reverse the valid data bit order.

At this point, the data format control is completed. The data after format control will be sent sequentially from high to low. Figure [46.9-1](#) shows the complete process of TX data format control.

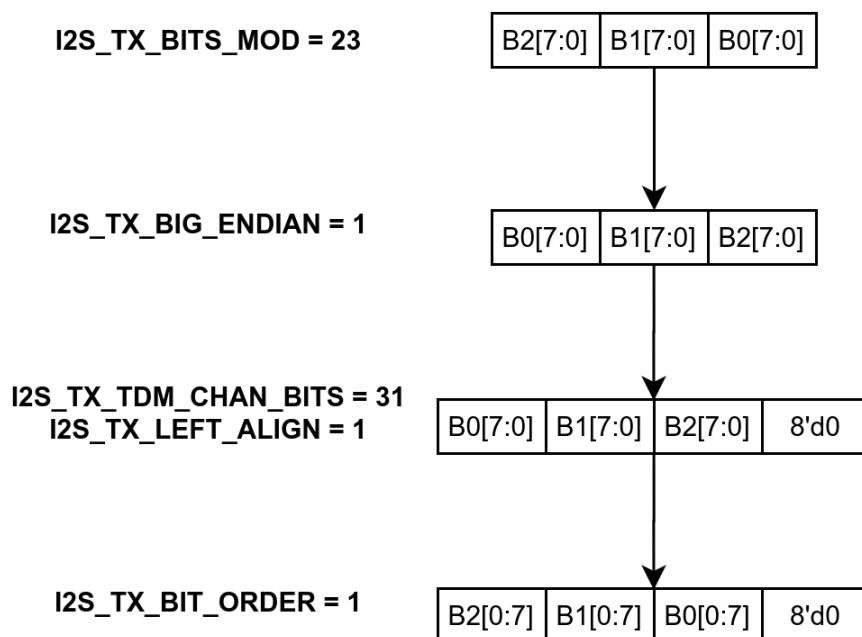


Figure 46.9-1. TX Data Format Control

46.9.2 Channel Mode Control

ESP32-P4 I2S_n supports both TDM TX mode and PDM TX mode. Set `I2S_TX_TDM_EN` to enable TDM TX mode, or set `I2S_TX_PDM_EN` to enable PDM TX mode.

Note:

`I2S_TX_TDM_EN` and `I2S_TX_PDM_EN` must not be cleared or set simultaneously.

46.9.2.1 TDM TX Mode

In TDM TX mode, I2S_n supports up to 16 channels to transmit data. The total number of TX channels in use is controlled by `I2S_TX_TDM_TOT_CHAN_NUM`. For example, If `I2S_TX_TDM_TOT_CHAN_NUM` is set to 5, six channels in total (channel 0 ~ 5) will be used to transmit data. See Figure 46.9-2.

In these TX channels, if `I2S_TX_TDM_CHANn_EN` is set to:

- 1: This channel transmits the channel data out;
- 0: The TX data to be sent by this channel is controlled by `I2S_TX_CHAN_EQUAL`:
 - 1: The data of the previous channel is sent out;
 - 0: The data stored in `I2S_SINGLE_DATA` is sent out.

Note:

Most stereo I2S_n codecs can be controlled by setting the I2S_n module into 2-channel mode under TDM standard.

In TDM TX master mode, WS signal is controlled by `I2S_TX_WS_IDLE_POL` and `I2S_TX_TDM_WS_WIDTH`:

- [I2S_TX_WS_IDLE_POL](#): The default level of WS signal;
- [I2S_TX_TDM_WS_WIDTH](#): The cycles the WS default level lasts for when transmitting all channel data.

[I2S_TX_HALF_SAMPLE_BITS](#) x 2 is equal to the BCK cycles in one WS period.

TDM Channel Configuration Example

In this example, the register configuration is as follows:

- [I2S_TX_TDM_CHAN_NUM](#) = 5, i.e., channel 0 ~ 5 are used to transmit data.
- [I2S_TX_CHAN_EQUAL](#) = 1, i.e., data of the previous channel will be transmitted if the bit [I2S_TX_TDM_CHAN \$n\$ _EN](#) (n = 0 ~ 5) is cleared.
- [I2S_TX_TDM_CHAN0/2/5_EN](#) = 1, i.e., these channels transmit their channel data out.
- [I2S_TX_TDM_CHAN1/3/4_EN](#) = 0, i.e., these channels transmit the previous channel's data out.

Once the configuration is done, data is transmitted as follows.

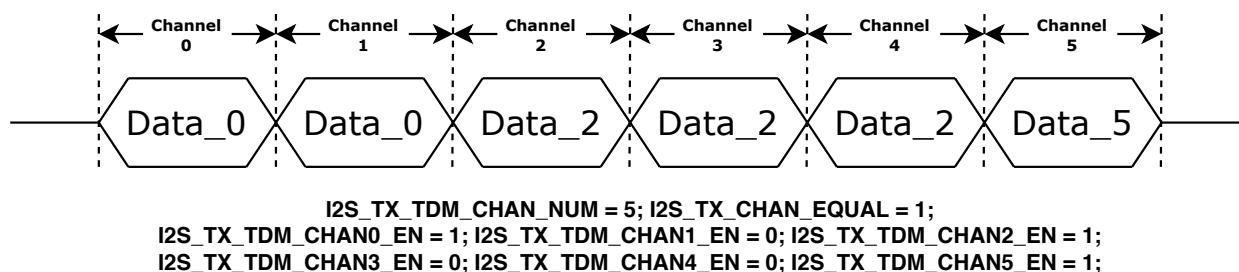


Figure 46.9-2. TDM Channel Control

46.9.2.2 PDM TX Mode

In PDM TX mode, [I2S \$n\$](#) supports both PDM raw data transmission and PCM-to-PDM data format conversion (the latter only supported by [I2S0](#)).

In PDM TX mode, fetching data through GDMA is controlled by [I2S_TX_MONO](#) and [I2S_TX_MONO_FST_VLD](#). See Table 46.9-3. Configure the two bits according to the data stored in memory, be it the single-channel or dual-channel data.

Table 46.9-3. Data-Fetching Control in PDM Mode

Data-Fetching Control Option	Mode	I2S_TX_MONO	I2S_TX_MONO_FST_VLD
Post data-fetching request to GDMA at any edge of WS signal	Stereo mode	0	x
Post data-fetching request to GDMA only at the second half period of WS signal	Mono mode	1	0
Post data-fetching request to GDMA only at the first half period of WS signal	Mono mode	1	1

When [I2S \$n\$](#) is in PDM TX master mode, the default level of WS signal is controlled by [I2S_TX_WS_IDLE_POL](#), and the WS signal frequency is half of the BCK signal frequency. The configuration of WS signal is similar to that of BCK signal. Please refer to Section 46.6 and Figure 46.6.

When **transmitting raw PDM data**, the I2S n channel mode is controlled by [I2S_TX_CHAN_MOD](#) and [I2S_TX_WS_IDLE_POL](#). See the table below.

Table 46.9-4. I2S n Channel Control for Raw PDM Data

Channel Control Option	Left Channel	Right Channel	Mode Control Field ¹	Channel Select Bit ²
Stereo mode	Transmit the left channel data	Transmit the right channel data	0	x
Mono mode	Transmit the left channel data	Transmit the left channel data	1	0
	Transmit the right channel data	Transmit the right channel data	1	1
	Transmit the right channel data	Transmit the right channel data	2	0
	Transmit the left channel data	Transmit the left channel data	2	1
	Transmit the value of “single” ³	Transmit the right channel data	3	0
	Transmit the left channel data	Transmit the value of “single”	3	1
	Transmit the left channel data	Transmit the value of “single”	4	0
	Transmit the value of “single”	Transmit the right channel data	4	1

¹ [I2S_TX_CHAN_MOD](#)

² [I2S_TX_WS_IDLE_POL](#)

³ The “single” value is equal to the value of [I2S_SINGLE_DATA](#).

If the **PCM-to-PDM converter is enabled** for I2S O , the PCM data through GDMA is converted to PDM data and then output in PDM signal format. Configure [I2S_PCM2PDM_CONV_EN](#) to enable the converter. The register configuration for the PCM-to-PDM converter is as follows:

- Configure 1-line PDM output format or 1-/2-line DAC output mode as the table below:

Table 46.9-5. I2S O PCM-to-PDM Data Output

Channel Output Format	I2S_TX_PDM_DAC_MODE_EN	I2S_TX_PDM_DAC_2OUT_EN
1-line PDM output format ¹	0	x
1-line DAC output format ²	1	0
2-line DAC output format	1	1

Note:

- In PDM output format, SD data of two channels is sent out in one WS period.
- In DAC output format, SD data of one channel is sent out in one WS period.
- 1-line PDM/DAC output format uses I2S nO _Data_out as the data output line.
- 2-line DAC output format uses I2S nO _Data_out and I2S nO _Data1_out as data output lines.
- In terms of application, the PDM output format is targeted at applications that require clock signals and can decode PDM format codecs, while the DAC output format is targeted at applications that do not rely on clock signals, do not have PDM format codecs, and can recover analog waveforms directly through low-pass filtering.

- Configure sampling frequency and upsampling rate as below:

When the I2S⁰ PCM-to-PDM converter is enabled, PDM clock frequency is equal to BCK frequency. The relation of sampling frequency (f_{Sampling} , the sampling frequency of PCM data) and BCK frequency (the sampling frequency of PDM data) is as follows:

$$f_{\text{Sampling}} = \frac{f_{\text{BCK}}}{\text{OSR}}$$

Upsampling rate (OSR) is related to `I2S_TX_PDM_SINC_OSR2` as follows:

$$\text{OSR} = \text{I2S_TX_PDM_SINC_OSR2} \times 64$$

Sampling frequency f_{Sampling} is related to `I2S_TX_PDM_FS` as follows:

$$f_{\text{Sampling}} = \text{I2S_TX_PDM_FS} \times 100$$

Configure the registers according to the needed sampling frequency, upsampling rate, and PDM clock frequency.

PDM Channel Configuration Example

In this example of I2S⁰, the register configuration is as follows.

- `I2S_PCM2PDM_CONV_EN` = 0, i.e., transmit the raw PDM data.
- `I2S_TX_MONO` = 0, i.e., data is fetched from memory via GDMA in both the high and low levels of WS.
- `I2S_TX_CHAN_MOD` = 2, i.e., mono mode is selected.
- `I2S_TX_WS_IDLE_POL` = 1, i.e., both the left and right channels transmit the left channel data, and the right channel data will be discarded.

Once the configuration is done, assume that the data in memory **after data format control** is:

Left	Right	Left	Right	...	Left	Right
------	-------	------	-------	-----	------	-------

Note:

1. The data above refers to the processed data after data format control instead of the original data.
2. “Left” and “Right” represent channel data, and their bit widths are channel valid data width. Please refer to Section 46.9.1.

Then the channel data is transmitted after channel mode control as follows.

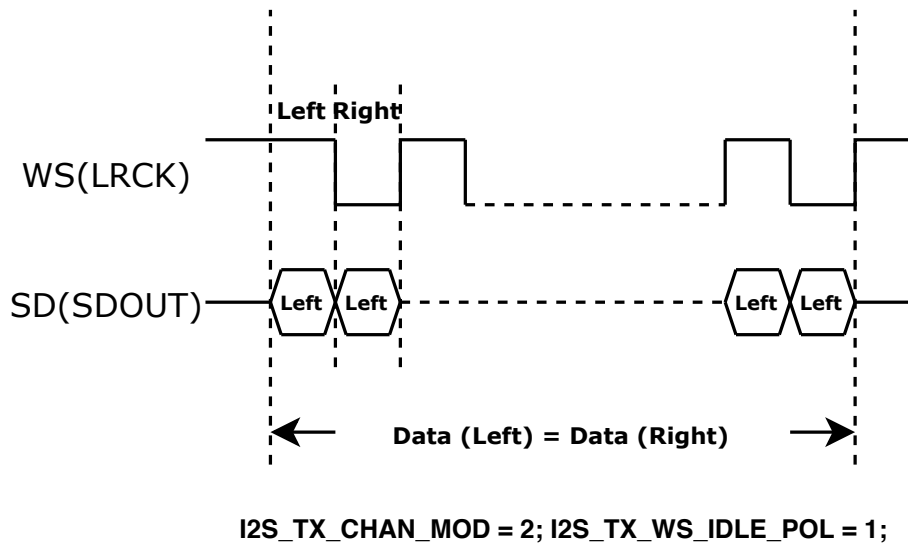


Figure 46.9-3. PDM Channel Control Example

46.9.3 Synchronous Counter

ESP32-P4 I2S_n TX unit contains synchronous counters to reflect the current I2S_n data transmission status. The counters can be used to detect I2S_n transmit data errors or to synchronize data when multiple devices are using I2S_n for data transmission.

ESP32-P4 I2S_n contains two types of synchronous counters:

- [I2S_TX_FIFO_CNT](#)
 - Each time the I2S_n TX FIFO reads data, the counter is incremented by 1.
 - The counter can be reset by setting [I2S_TX_FIFO_CNT_RST](#).
 - The counter is 31-bit wide and is automatically reset to zero on overflow.
- [I2S_TX_BCK_CNT](#)
 - Each time I2S_n TX transmits a BCK cycle, the counter is incremented by 1.
 - The counter can be reset by setting [I2S_TX_BCK_CNT_RST](#).
 - The counter is 31-bit wide and is automatically reset to zero on overflow.

Note:

- Each time the I2S_n TX FIFO transmits data, a channel data is sent. Therefore, when I2S_n transmits the data correctly, $I2S_TX_BCK_CNT = I2S_TX_FIFO_CNT \times \text{the bit width of channel TX data}$ (see Section 46.9.1.4).
- When the I2S_n TX unit transmits the data, there is some delay between fetching data from the FIFO and transmitting it to the line, so the value of [I2S_TX_BCK_CNT](#) might be a little different from that of [I2S_TX_FIFO_CNT](#).

Application scenarios

1. I2S TX synchronization for multiple devices

When multiple devices transmit data through the I2S TX at the same time, desynchronization may occur after a long period of time due to small frequency differences in the clock sources. In this case, read the

`I2S_TX_FIFO_CNT` value of each device to determine the location of the audio data being sent to perform synchronization.

2. I2S TX stability check

When the device is in an application scenario with high GDMA usage, there may be numbers missing in I2S TX occasionally due to multiplexing arbitration. In this case, check the ratio of `I2S_TX_BCK_CNT` to `I2S_TX_FIFO_CNT` to confirm the stability of the device when running I2S TX.

46.10 Receiving Data

In RX mode, I2S_n first reads data from the peripheral interface and then stores the data in memory via GDMA according to the configured channel mode and data mode.

46.10.1 Channel Mode Control

ESP32-P4 I2S_n supports both TDM RX mode and PDM RX mode. Set `I2S_RX_TDM_EN` to enable TDM RX mode, or set `I2S_RX_PDM_EN` to enable PDM RX mode.

Note:

`I2S_RX_TDM_EN` and `I2S_RX_PDM_EN` must not be cleared or set simultaneously.

46.10.1.1 TDM RX Mode

In TDM RX mode, I2S_n supports up to 16 channels to input data. The total number of RX channels in use is controlled by `I2S_RX_TDM_TOT_CHAN_NUM`. For example, if `I2S_RX_TDM_TOT_CHAN_NUM` is set to 5, channel 0 ~ 5 will be used to receive data.

In these RX channels, if `I2S_RX_TDM_CHANn_EN` is set to:

- 1: The channel data is valid and will be stored into RX FIFO;
- 0: The channel data is invalid and will not be stored into RX FIFO.

In TDM master mode, WS signal is controlled by `I2S_RX_WS_IDLE_POL` and `I2S_RX_TDM_WS_WIDTH`.

- `I2S_RX_WS_IDLE_POL`: The default level of WS signal;
- `I2S_RX_TDM_WS_WIDTH`: The cycles the WS default level lasts for when receiving all channel data.

`I2S_RX_HALF_SAMPLE_BITS` x 2 is equal to the BCK cycles in one WS period.

46.10.1.2 PDM RX Mode

In PDM RX mode, I2S_n supports both PDM raw data reception and PDM-to-PCM data format conversion (the latter only supported by I2S₀).

In PDM RX mode, I2S_n converts the serial data from channels to the data to be entered into memory.

In PDM RX master mode, the default level of the WS signal is controlled by `I2S_RX_WS_IDLE_POL`. WS frequency is half of the BCK frequency. The configuration of the BCK signal is similar to that of WS signal as described in Section 46.6. Note, in PDM RX mode, the value of `I2S_RX_HALF_SAMPLE_BITS` must be same as that of `I2S_RX_BITS_MOD`.

When the **PDM-to-PCM converter is enabled** for I2S0, the received PDM data is converted to PCM data and controlled according to the data mode. Configure [I2S_RX_PDM2PCM_EN](#) to enable this converter. The register configuration for PDM-to-PCM converter is as follows:

- Configure sampling frequency and downsampling rate.

When I2S0 PDM-to-PCM converter is enabled, PDM clock frequency is:

- in master mode: PDM clock frequency is equal to BCK frequency.
- in slave mode: PDM clock is provided by external device.

The sampling frequency (f_{Sampling}) is related to PDM clock frequency as follows:

$$f_{\text{Sampling}} = \frac{f_{\text{PDM}}}{\text{DSR}}$$

Downsampling rate (DSR) is related to [I2S_RX_PDM_SINC_DSR_16_EN](#) as follows:

$$\text{DSR} = \text{I2S_RX_PDM_SINC_DSR_16_EN} \times 64$$

Configure the registers according to needed master/slave mode, sampling frequency, and downsampling rate.

- Configure valid channels.

When the PDM-to-PCM converter is enabled, input signals from eight channels are supported at most. See Table 46.10-1 for the register configuration and related channels.

Table 46.10-1. PDM-to-PCM Data Input

Input Data Signal	Channel	Enable Register
I2S0I_Data_in	Left channel	I2S_RX_TDM_PDM_CHAN0_EN
	Right channel	I2S_RX_TDM_PDM_CHAN1_EN
I2S0I1_Data_in	Left channel	I2S_RX_TDM_PDM_CHAN2_EN
	Right channel	I2S_RX_TDM_PDM_CHAN3_EN
I2S0I2_Data_in	Left channel	I2S_RX_TDM_PDM_CHAN4_EN
	Right channel	I2S_RX_TDM_PDM_CHAN5_EN
I2S0I3_Data_in	Left channel	I2S_RX_TDM_PDM_CHAN6_EN
	Right channel	I2S_RX_TDM_PDM_CHAN7_EN

46.10.2 Data Format Control

The data format of I2S0 is controlled in the following phases:

- Phase I: Serial input data is converted into the data to be saved to RX FIFO;
- Phase II: The data is read from RX FIFO and converted according to the input data mode.

46.10.2.1 Bit Order Control of Channel Data

The channel data will be stored as the data to be input in order from high to low. The data bit order in each channel is controlled by [I2S_RX_BIT_ORDER](#):

- 0: The bit order of the data to be input is not reversed;
- 1: The bit order of the data to be input is reversed.

At this point, the first phase of data format control is completed. The data to be input after bit order control is stored in the RX FIFO.

46.10.2.2 Bit Width Control of Channel Storage (Valid) Data

The storage data width in each channel is controlled by [I2S_RX_BITS_MOD](#) and [I2S_RX_24_FILL_EN](#). See the table below.

Table 46.10-2. Channel Storage Data Width

Channel Storage Data Width	I2S_RX_BITS_MOD	I2S_RX_24_FILL_EN
32	31	x
	23	1
24	23	0
16	15	x
8	7	x

46.10.2.3 Bit Width Control of Channel RX Data

The RX data width in each channel is determined by [I2S_RX_TDM_CHAN_BITS](#).

- If the storage data width in each channel is smaller than the received (RX) data width, then only the bits within the storage data width is saved into memory. Configure [I2S_RX_LEFT_ALIGN](#) to:
 - 0: Only the lower bits of the received data within the storage data width is stored to memory;
 - 1: Only the higher bits of the received data within the storage data width is stored to memory.
- If the received data width is smaller than the storage data width in each channel, the higher bits of the received data will be filled with zeros and then the data is saved to memory.

46.10.2.4 Endian Control of Channel Storage Data

The received data is then converted into storage data (to be stored to memory) after some processing, such as discarding extra bits or filling zeros in missing bits. The endian of the storage data is controlled by [I2S_RX_BIG_ENDIAN](#) under various data width. See the table below.

Table 46.10-3. Channel Storage Data Endian

Channel Storage Data Width	Original Data	Endian of Processed Data	I2S_RX_BIG_ENDIAN
32	{B3, B2, B1, B0}	{B3, B2, B1, B0}	0
		{B0, B1, B2, B3}	1
24	{B2, B1, B0}	{B2, B1, B0}	0
		{B0, B1, B2}	1
16	{B1, B0}	{B1, B0}	0
		{B0, B1}	1
8	{B0}	{B0}	x

46.10.2.5 A-law/ μ -law Compression and Decompression

ESP32-P4 I2S_n compresses/decompresses the storage data in 32-bit by A-law or by μ -law. By default, zeros are filled into high bits.

Configure [I2S_RX_PCM_BYPASS](#):

- 0: Compress or decompress the data
- 1: Do not compress or decompress the data

Configure [I2S_RX_PCM_CONF](#):

- 0: Decompress the data using A-law
- 1: Compress the data using A-law
- 2: Decompress the data using μ -law
- 3: Compress the data using μ -law

At this point, the data format control is completed. Data then is stored into memory via GDMA.

46.11 Event Task Matrix Feature

ESP32-P4 I2S_n supports the Event Task Matrix (ETM) function, which allows I2S_n's ETM tasks to be triggered by any peripherals' ETM events, or I2S_n's ETM events to trigger any peripherals' ETM tasks. This section introduces the ETM tasks and events related to I2S_n. For more information, please refer to [Chapter 13 Event Task Matrix \(ETM\)](#).

I2S_n can receive the following ETM tasks:

- I2S_n_TASK_START_TX: Enables I2S_n TX for data transfer.
- I2S_n_TASK_START_RX: Enables I2S_n RX for data transfer.
- I2S_n_TASK_STOP_TX: Stops I2S_n TX data transfer.
- I2S_n_TASK_STOP_RX: Stops I2S_n RX data transfer.

I2S_n can generate the following ETM events:

- I2S_n_EVT_TX_DONE: Indicates that I2S_n TX has completed data transmission.
- I2S_n_EVT_RX_DONE: Indicates that I2S_n RX has completed data reception.
- I2S_n_EVT_X_WORDS_SENT: Indicates that the word number sent by I2S_n TX is equal to or larger than the value set by [I2S_ETM_TX_SEND_WORD_NUM](#).
- I2S_n_EVT_X_WORDS_RECEIVED: Indicates that the word number received by I2S_n RX is equal to or larger than the value set by [I2S_ETM_RX_RECEIVE_WORD_NUM](#).

In practical applications, I2S_n's ETM events can trigger its own ETM tasks. For example, the I2S_n_EVT_X_WORDS_SENT event can trigger the I2S_n_TASK_STOP_TX task, and in this way stop the I2S_n operation through ETM.

46.12 I2S Interrupts

ESP32-P4's I2S_n can generate the I2S_n_INTR interrupt signal that will be sent to the [Interrupt Matrix](#). There are several internal interrupt sources from I2S_n that can generate the above interrupt signal(s) as follows:

- I2S_n_TX_HUNG_INT: Triggered when data transmission is timed out. For example, if the I2S_n module is configured as TX slave mode, but the master does not provide BCK or WS signals for a long time (specified in [I2S_LC_HUNG_CONF_REG](#)), this interrupt will be triggered.
- I2S_n_RX_HUNG_INT: Triggered when the data reception is timed out. For example, if the I2S_n module is configured as RX slave mode, but the master does not transmit data for a long time (specified in [I2S_LC_HUNG_CONF_REG](#)), this interrupt will be triggered.
- I2S_n_TX_DONE_INT: Triggered when data transmission is completed.
- I2S_n_RX_DONE_INT: Triggered when the data reception is completed.

Note:

For definitions of *interrupt*, *interrupt signal*, *interrupt source*, and their correlations, please refer to Chapter [12 Interrupt Matrix](#) > Section [12.2 Interrupt Terminology in ESP32-P4](#).

Each interrupt source can be configured by a common set of registers that are described in Section [Interrupt Configuration Registers](#). The specific registers can be found in Section [46.14 Register Summary](#).

46.13 Software Configuration Process

46.13.1 Configure I2S as TX Mode

Follow the steps below to configure I2S_n as TX mode via software:

1. Configure the clock as described in Section [46.6](#).
2. Configure signal pins according to Table [46.4-1](#).
3. Select the mode needed by configuring [I2S_TX_SLAVE_MOD](#).
 - 0: Master TX mode
 - 1: Slave TX mode
4. Set needed TX data mode and TX channel mode as described in Section [46.9](#), and then set [I2S_TX_UPDATE](#).
5. Reset TX unit and TX FIFO as described in Section [46.7](#).
6. Enable corresponding interrupts. See Section [46.12](#).
7. Configure GDMA outlink.
8. Set [I2S_TX_STOP_EN](#) if needed. For more information, please refer to Section [46.8.1](#).
9. Start transmitting data:
 - In master mode, wait till I2S_n slave gets ready, then set [I2S_TX_START](#) to start transmitting data;

- In slave mode, set [I2S_TX_START](#). When the I2S_n master supplies BCK and WS signals, I2S_n slave starts transmitting data.
10. Wait for the interrupt signals set in Step 6, or check whether the transfer is completed by querying [I2S_TX_IDLE](#):
 - 0: Transmitter is working;
 - 1: Transmitter is in idle state.
 11. Clear [I2S_TX_START](#) to stop data transfer.

46.13.2 Configure I2S as RX Mode

Follow the steps below to configure I2S_n as RX mode via software:

1. Configure the clock as described in Section 46.6.
2. Configure signal pins according to Table 46.4-1.
3. Select the mode needed by configuring [I2S_RX_SLAVE_MOD](#).
 - 0: Master RX mode
 - 1: Slave RX mode
4. Set needed RX data mode and RX channel mode as described in Section 46.10, and then set [I2S_RX_UPDATE](#).
5. Reset the RX unit and its FIFO according to Section 46.7.
6. Enable the corresponding interrupts. See Section 46.12.
7. Configure GDMA inlink and set the length of RX data by configuring [I2S_RX_EOF_NUM_REG](#).
8. Start receiving data:
 - In master mode, when the slave is ready, set [I2S_RX_START](#) to start receiving data.
 - In slave mode, set [I2S_RX_START](#) to start receiving data when BCK and WS signals are received from the master.
9. The received data is then stored to the specified memory address according to the configuration of GDMA. Then the corresponding interrupt set in step 6 is generated.

46.14 Register Summary

The addresses in this section are relative to I2S_n base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
Interrupt registers			
I2S_INT_RAW_REG	I2S interrupt raw register	0x000C	R/SS/WTC
I2S_INT_ST_REG	I2S interrupt status register	0x0010	RO
I2S_INT_ENA_REG	I2S interrupt enable register	0x0014	R/W
I2S_INT_CLR_REG	I2S interrupt clear register	0x0018	WT
RX control and configuration registers			
I2S_RX_CONF_REG	I2S RX configuration register	0x0020	varies
I2S_RX_CONF1_REG	I2S RX configuration register 1	0x0028	R/W
I2S_RX_PDM2PCM_CONF_REG	I2S RX PDM-to-PCM configuration register	0x0048	R/W
I2S_RX_TDM_CTRL_REG	I2S TX TDM mode control register	0x0050	R/W
I2S_RXEOF_NUM_REG	I2S RX data number control register	0x0064	R/W
TX control and configuration registers			
I2S_TX_CONF_REG	I2S TX configuration register	0x0024	varies
I2S_TX_CONF1_REG	I2S TX configuration register 1	0x002C	R/W
I2S_TX_PCM2PDM_CONF_REG	I2S TX PCM-to-PDM configuration register (for I2S ₀ only)	0x0040	R/W
I2S_TX_PCM2PDM_CONF1_REG	I2S TX PCM-to-PDM configuration register (for I2S ₀ only)	0x0044	R/W
I2S_TX_TDM_CTRL_REG	I2S TX TDM mode control register	0x0054	R/W
RX clock and timing register			
I2S_RX_TIMING_REG	I2S RX timing control register (some fields for I2S ₀ only)	0x0058	R/W
TX clock and timing register			
I2S_TX_TIMING_REG	I2S TX timing control register (some fields for I2S ₀ only)	0x005C	R/W
Control and configuration registers			
I2S_LC_HUNG_CONF_REG	I2S timeout configuration register	0x0060	R/W
I2S_CONF_SIGLE_DATA_REG	I2S single data register	0x0068	R/W
TX status register			
I2S_STATE_REG	I2S TX status register	0x006C	RO
ETM register			
I2S_ETM_CONF_REG	I2S ETM configuration register	0x0070	R/W
Sync counter registers			
I2S_FIFO_CNT_REG	I2S sync counter register	0x0074	varies
I2S_BCK_CNT_REG	I2S sync counter register	0x0078	varies
Clock registers			
I2S_CLK_GATE_REG	Clock gate register	0x007C	R/W

Name	Description	Address	Access
Version register			
I2S_DATE_REG	Version control register	0x0080	R/W

46.15 Registers

The addresses in this section are relative to I2S_n base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

For how to program reserved fields, please refer to Section *Programming Reserved Register Field*.

Register 46.1. I2S_INT_RAW_REG (0x000C)

(reserved)																																I2S_TX_HUNG_INT_RAW I2S_RX_HUNG_INT_RAW I2S_TX_DONE_INT_RAW I2S_RX_DONE_INT_RAW																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																	
31																															4	3	2	1	0	Reset																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																													
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

I2S_RX_DONE_INT_RAW The raw interrupt status of the [I2S_RX_DONE_INT](#) interrupt. (R/SS/WTC)

I2S_TX_DONE_INT_RAW The raw interrupt status of the [I2S_TX_DONE_INT](#) interrupt.
(R/SS/WTC)

I2S_RX_HUNG_INT_RAW The raw interrupt status of the [I2S_RX_HUNG_INT](#) interrupt. (R/SS/WTC)

I2S_TX_HUNG_INT_RAW The raw interrupt status of the [I2S_TX_HUNG_INT](#) interrupt. (R/SS/WTC)

Register 46.2. I2S_INT_ST_REG (0x0010)

(reserved)																										I2S_TX_HUNG_INT_ST I2S_RX_HUNG_INT_ST I2S_TX_DONE_INT_ST I2S_RX_DONE_INT_ST					
31																										4	3	2	1	0	Reset
0 0																										0	0	0	0		

I2S_RX_DONE_INT_ST The masked interrupt status of the [I2S_RX_DONE_INT](#) interrupt. (RO)

I2S_TX_DONE_INT_ST The masked interrupt status of the [I2S_TX_DONE_INT](#) interrupt. (RO)

I2S_RX_HUNG_INT_ST The masked interrupt status of the [I2S_RX_HUNG_INT](#) interrupt. (RO)

I2S_TX_HUNG_INT_ST The masked interrupt status of the [I2S_TX_HUNG_INT](#) interrupt. (RO)

Register 46.3. I2S_INT_ENA_REG (0x0014)

(reserved)																												I2S_TX_HUNG_INT_ENA I2S_RX_HUNG_INT_ENA I2S_TX_DONE_INT_ENA I2S_RX_DONE_INT_ENA					
31																												4	3	2	1	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

I2S_RX_DONE_INT_ENA Write 1 to enable the [I2S_RX_DONE_INT](#) interrupt. (R/W)

I2S_TX_DONE_INT_ENA Write 1 to enable the [I2S_TX_DONE_INT](#) interrupt. (R/W)

I2S_RX_HUNG_INT_ENA Write 1 to enable the [I2S_RX_HUNG_INT](#) interrupt. (R/W)

I2S_TX_HUNG_INT_ENA Write 1 to enable the [I2S_TX_HUNG_INT](#) interrupt. (R/W)

Register 46.4. I2S_INT_CLR_REG (0x0018)

(reserved)																												I2S-TX_HUNG_INT_CLR I2S-RX_HUNG_INT_CLR I2S-TX_DONE_INT_CLR I2S-RX_DONE_INT_CLR				
31																												4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

I2S_RX_DONE_INT_CLR Write 1 to clear the [I2S_RX_DONE_INT](#) interrupt. (WT)

I2S_TX_DONE_INT_CLR Write 1 to clear the [I2S_TX_DONE_INT](#) interrupt. (WT)

I2S_RX_HUNG_INT_CLR Write 1 to clear the [I2S_RX_HUNG_INT](#) interrupt. (WT)

I2S_TX_HUNG_INT_CLR Write 1 to clear the [I2S_TX_HUNG_INT](#) interrupt. (WT)

Register 46.5. I2S_RX_CONF_REG (0x0020)

(reserved)						I2S_RX_BCK_DIV_NUM				I2S_RX_PDM_EN		I2S_RX_TDM_EN		I2S_RX_BIT_ORDER		I2S_RX_WS_IDLE_POL		I2S_RX_24_FILL_EN		(reserved)		I2S_RX_LEFT_ALIGN		I2S_RX_PCM_BYPASS		I2S_RX_PCM_CONF		I2S_RX_MONO_FST_VLD		I2S_RX_UPDATE		I2S_RX_BIG_ENDIAN		I2S_RX_MONO		I2S_RX_STOP_MODE		I2S_RX_SLAVE_MOD		I2S_RX_START		I2S_RX_FIFO_RES	
31	27					26	21					20	19	18	17	16	15	14	12	11	10	9	8	7	6	5	4	3	2	1	0												
0	0	0	0	0	0	6					0	0	0	0	0	0	1	0	1	0x1	1	0	0	0	0	0	0	0	0	0	0	0	0	Reset									

I2S_RX_RESET Configures whether to reset RX unit.

0: No effect

1: Reset

(WT)

I2S_RX_FIFO_RESET Configures whether to reset RX FIFO.

0: No effect

1: Reset

(WT)

I2S_RX_START Configures whether to start receiving data.

0: No effect

1: Start

(R/W/SC)

I2S_RX_SLAVE_MOD Configures whether to enable slave RX mode.

0: Enable master mode

1: Enable slave mode

(R/W)

I2S_RX_STOP_MODE Configures when I2S RX stop data reception.

0: Only stops when I2S_RX_START is cleared

1: Stops when `I2S_RX_START` is cleared or the number of received bytes is greater than the value configured in `I2S_RX_EOF_NUM_REG`

2: Stops when I2S_RX_START is cleared or GDMA RX FIFO is full

(R/W)

I2S_RX_MONO Configures whether to enable RX unit in mono mode.

0: Disable

1: Enable

(R/W)

I2S_RX_BIG_ENDIAN Configures I2S RX byte endian.

0: Low address data is saved to low address

1: Low address data is saved to high address

(R/W)

Continued on the next page...

Register 46.5. I2S_RX_CONF_REG (0x0020)

Continued from the previous page...

I2S_RX_UPDATE Configures whether to update I2S RX registers from APB clock domain to I2S RX clock domain. This bit will be cleared by hardware after the register update is done.

0: No effect

1: Update

(R/W/SC)

I2S_RX_MONO_FST_VLD Configures the valid data channel in I2S RX mono mode.

0: The second channel data valid

1: The first channel data valid

(R/W)

I2S_RX_PCM_CONF Configures I2S RX compress/decompress mode.

0 (atol): A-law decompress

1 (ltoa): A-law compress

2 (utol): μ -law decompress

3 (ltou): μ -law compress

(R/W)

I2S_RX_PCM_BYPASS Configures whether to bypass the Compress/Decompress units for received data.

0: No effect

1: Bypass

(R/W)

I2S_RX_MSB_SHIFT Configures the timing between the WS signal and the MSB of data.

0: Align at the rising edge

1: The WS signal changes one BCK clock earlier

(R/W)

I2S_RX_LEFT_ALIGN Configures I2S RX alignment mode.

0: Right alignment mode

1: Left alignment mode

(R/W)

Continued on the next page...

Register 46.5. I2S_RX_CONF_REG (0x0020)

Continued from the previous page...

I2S_RX_24_FILL_EN Configures the bit number that the 24-bit channel data is stored to.

0: Store 24-bit channel data to 24 bits

1: Store 24-bit channel data to 32 bits (Extra bits are filled with zeros)

(R/W)

I2S_RX_WS_IDLE_POL Configures the relationship between WS level and which channel data to receive.

0: WS remains low when receiving left channel data and high when receiving right channel data

1: WS remains high when receiving left channel data and low when receiving right channel data

(R/W)

I2S_RX_BIT_ORDER Configures whether to reverse the bit order of the I2S RX data to be received.

0: Not reverse

1: Reverse

(R/W)

I2S_RX_TDM_EN Configures whether to enable I2S TDM RX mode.

0: Disable

1: Enable

(R/W)

I2S_RX_PDM_EN Configures whether to enable I2S PDM RX mode.

0: Disable

1: Enable

(R/W)

I2S_RX_BCK_DIV_NUM Configures the divider of BCK in RX mode. Note this divider must not be configured to 1. (R/W)

Register 46.6. I2S_RX_CONF1_REG (0x0028)

31		27		26		19		18		14		13		9		8		0	
I2S_RX_TDM_CHAN_BITS				I2S_RX_HALF_SAMPLE_BITS				I2S_RX_BITS_MOD				(reserved)				I2S_RX_TDM_WS_WIDTH			
0xf				0xf				0xf				0 0 0 0 0				0x0			

Reset

I2S_RX_TDM_WS_WIDTH Configures the width of I2S_{nl}_WS_out (WS default level) at idle level in TDM mode. Width of I2S_{nl}_WS_out at idle level in TDM mode = (I2S_RX_TDM_WS_WIDTH[8:0] + 1) x T_BCK. (R/W)

I2S_RX_BITS_MOD Configures the valid data bit length of I2S RX channel.

- 7: All the valid channel data is in 8-bit mode
15: All the valid channel data is in 16-bit mode
23: All the valid channel data is in 24-bit mode
31: All the valid channel data is in 32-bit mode
Other values are invalid.
(R/W)

I2S_RX_HALF_SAMPLE_BITS Configures I2S RX sample bits. BCK cycles in one WS period = I2S_RX_HALF_SAMPLE_BITS x 2. (R/W)

I2S_RX_TDM_CHAN_BITS Configures RX bit number for each channel in TDM mode. Bit number expected = I2S_RX_TDM_CHAN_BITS + 1. (R/W)

Register 46.7. I2S_RX_PDM2PCM_CONF_REG (for I2S0 only) (0x0048)

[illegible]

I2S_RX_PDM2PCM_EN Configures whether to enable the PDM-to-PCM converter in RX mode.

0: Disable

1: Enable

(R/W)

I2S_RX_PDM_SINC_DSR_16_EN Configures the downsampling rate of PDM RX filter group 1 module.

0: 64

1: 128

(R/W)

I2S_RX_PDM2PCM_AMPLIFY_NUM Configures the PDM-to-PCM RX amplify number. PCM data will be multiplied by this value before outputting.

(R/W)

I2S_RX_PDM_HP_BYPASS Configures whether PDM-to-PCM RX bypasses the HP filter.

0: Not bypass

1: Bypass

(R/W)

Register 46.8. I2S_RX_TDM_CTRL_REG (0x0050)

[illegible]

I2S_RX_TDM_PDM_CHAN n _EN (n : 0-7) Configures whether to enable the valid data input of I2S RX TDM or PDM channel n .

0: Disable. Channel *n* only inputs 0

1: Enable

(R/W)

I2S_RX_TDM_CHAN n _EN (n : 8-15) Configures whether to enable the valid data input of I2S RX TDM channel n .

0: Disable. Channel *n* only inputs 0

1: Enable

(R/W)

I2S_RX_TDM_TOT_CHAN_NUM Configures the total number of channels in use in I2S RX TDM mode. Total channel number in use = I2S_RX_TDM_TOT_CHAN_NUM + 1. (R/W)

Register 46.9. I2S_RXEOF_NUM_REG (0x0064)

(reserved)	I2S_RX_EOF_NUM
31	0
12	11
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0x40
	Reset

I2S_RX_EOF_NUM Configures the bit length of RX data. Bit length of RX data = (I2S_RX_BITS_MOD + 1) x (I2S_RX_EOF_NUM + 1). Once the received data reaches such bit length, an **AHB_DMA_IN_SUC_EOF_CHn_INT** interrupt is triggered in the configured GDMA RX channel.
(R/W)

Register 46.10. I2S_TX_CONF_REG (0x0024)

(reserved)		I2S_SIG_LOOPBACK					I2S_TX_CHAN_MOD					I2S_TX_BCK_DIV_NUM					I2S_TX_PDM_EN					I2S_TX_TDM_EN					I2S_TX_BIT_ORDER					I2S_TX_WS_IDLE_POL					I2S_TX_24_FILL_EN					I2S_TX_LEFT_ALIGN					I2S_TX_BCK_MSB_SHIFT					I2S_TX_PCM_BYPASS					I2S_TX_PCM_CONF					I2S_TX_MONO_FST_VLD					I2S_TX_UPDATE					I2S_TX_BIG_ENDIAN					I2S_TX_MONO					I2S_TX_CHAN_EQUAL					I2S_TX_STOP_EN					I2S_TX_SLAVE_MOD					I2S_TX_FIFO_RESET																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																							
31	30	29	27	26	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Reset																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																													
0	0	0		6		0	0	0	0	0	1	1	1	1	0x0	1	0	0	0	0	0	1	0	0	0	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																														

I2S_TX_RESET Configures whether to reset TX unit.

0: No effect

1: Reset

(WT)

I2S_TX_FIFO_RESET Configures whether to reset TX FIFO.

0: No effect

1: Reset

(WT)

I2S_TX_START Configures whether to start transmitting data.

0: No effect

1: Start

(R/W/SC)

I2S_TX_SLAVE_MOD Configures whether to enable slave TX mode.

0: Enable master mode

1: Enable slave mode

(R/W)

I2S_TX_STOP_EN Configures whether to stop outputting the BCK signal and the WS signal when TX FIFO is empty.

0: No effect

1: Stop

(R/W)

I2S_TX_CHAN_EQUAL Configures whether to equalize left channel data and right channel data in I2S TX mono mode or TDM mode.

0: The I2S_SINGLE_DATA is invalid channel data in I2S TX mono mode or TDM mode

1: The left channel data is equal to right channel data in I2S TX mono mode or TDM mode

(R/W)

I2S_TX_MONO Configures whether to enable TX unit in mono mode.

0: Disable

1: Enable

(R/W)

Continued on the next page...

Register 46.10. I2S_TX_CONF_REG (0x0024)

Continued from the previous page...

I2S_TX_BIG_ENDIAN Configures I2S TX byte endian.

0: Low address with low address value

1: Low address value to high address

(R/W)

I2S_TX_UPDATE Configures whether to update I2S TX registers from APB clock domain to I2S TX clock domain.

0: No effect

1: Update

This bit will be cleared by hardware after update register done.

(R/W/SC)

I2S_TX_MONO_FST_VLD Configures the valid data channel in I2S TX mono mode.

0: The second channel data valid

1: The first channel data valid

(R/W)

I2S_TX_PCM_CONF Configures the I2S TX compress/decompress mode.

0 (atol): A-law decompress

1 (ltoa): A-law compress

2 (utol): μ -law decompress

3 (ltou): μ -law compress

(R/W)

I2S_TX_PCM_BYPASS Configures whether to bypass Compress/Decompress units for transmitted data.

0: No effect

1: Bypass

(R/W)

I2S_TX_MSB_SHIFT Configures the timing between the WS signal and the MSB of data.

0: Align at the rising edge

1: WS signal changes one BCK clock earlier

(R/W)

I2S_TX_BCK_NO_DLY Configures the source of the BCK rising and falling edges in master mode.

0: Rising and falling edges are constructed based on the BCK input from I2S_nO_BCK_in

1: Rising and falling edges are constructed by dividing the clock of I2S_n TX

(R/W)

I2S_TX_LEFT_ALIGN Configures I2S TX alignment mode.

0: Right alignment mode

1: Left alignment mode

(R/W)

Continued on the next page...

Register 46.10. I2S_TX_CONF_REG (0x0024)

Continued from the previous page...

I2S_TX_24_FILL_EN Configures the bit number that the 24 channel bits are stored to.

0: Store 24-bit channel data to 24 bits

1: Store 24-bit channel data to 32 bits (Extra bits are filled with zeros)

(R/W)

I2S_TX_WS_IDLE_POL Configures the relationship between WS and which channel data to transmit.

0: WS remains low when transmitting left channel data and high when transmitting right channel data

1: WS remains high when transmitting left channel data and low when transmitting right channel data

(R/W)

I2S_TX_BIT_ORDER Configures whether to reverse the bit order of valid data to be sent by the I2S TX.

0: Not reverse

1: Reverse

(R/W)

I2S_TX_TDM_EN Configures whether to enable I2S TDM TX mode.

0: Disable

1: Enable

(R/W)

I2S_TX_PDM_EN Configures whether to enable I2S PDM TX mode.

0: Disable

1: Enable

(R/W)

I2S_TX_BCK_DIV_NUM Configures the divider of BCK in TX mode. Note this divider must not be configured to 1. (R/W)

I2S_TX_CHAN_MOD Configures I2S TX channel mode. For more information, see Table [46.9-4](#).

(R/W)

I2S_SIG_LOOPBACK Configures whether to enable TX unit and RX unit sharing the same WS and BCK signals.

0: Disable

1: Enable

(R/W)

Register 46.11. I2S_TX_CONF1_REG (0x002C)

I2S_TX_TDM_CHAN_BITS								I2S_TX_HALF_SAMPLE_BITS								I2S_TX_BITS_MOD								(reserved)								I2S_TX_TDM_WS_WIDTH							
31	27	26	19	18	14	13	9	8	0																														
0xf		0xf				0xf		0	0	0	0	0	0x0				Reset																						

I2S_TX_TDM_WS_WIDTH Configures the width of I2S_nO_WS_out (WS default level) at idle level in TDM mode. The width of I2S_nO_WS_out at idle level in TDM mode = (I2S_TX_TDM_WS_WIDTH[8:0] + 1) x T_BCK. (R/W)

I2S_TX_BITS_MOD Configures the valid data bit length of I2S TX channel.

- 7: All the valid channel data is in 8-bit mode
- 15: All the valid channel data is in 16-bit mode
- 23: All the valid channel data is in 24-bit mode
- 31: All the valid channel data is in 32-bit mode
- Other values are invalid.
- (R/W)

I2S_TX_HALF_SAMPLE_BITS Configures I2S TX sample bits. BCK cycles in one WS period = I2S_TX_HALF_SAMPLE_BITS x 2. (R/W)

I2S_TX_TDM_CHAN_BITS Configures TX bit number for each channel in TDM mode. Bit number expected = I2S_TX_TDM_CHAN_BITS + 1. (R/W)

Register 46.12. I2S_TX_PCM2PDM_CONF_REG (for I2S0 only) (0x0040)

(reserved)							I2S_PCM2PDM_CONV_EN				I2S_TX_PDM_DAC_MODE_EN				I2S_TX_PDM_DAC_2OUT_EN				(reserved)							I2S_TX_PDM_SINC_OSR2				(reserved)	
31	26					25	24	23	22	21	20	19	18	17	16	15	14	13	12	5				4	1		0				
0	0	0	0	0	0	0	0	0	1	0	0x1	0x1	0x1	0x1	0x0				0x2				0		Reset						

I2S_TX_PDM_SINC_OSR2 Configures I2S TX PDM OSR value. (R/W)

I2S_TX_PDM_DAC_2OUT_EN Configures whether to enable I2S TX PDM DAC mode.

0: Enable 1-line DAC output mode

1: Enable 2-line DAC output mode

Only valid when I2S_TX_PDM_DAC_MODE_EN is set.

(R/W)

I2S_TX_PDM_DAC_MODE_EN Configures whether to enable 1-line PDM output mode or DAC output mode.

0: Enable 1-line PDM output mode

1: Enable DAC output mode

(R/W)

I2S_PCM2PDM_CONV_EN Configures whether to enable the I2S TX PCM-to-PDM converter.

0: Disable

1: Enable

(R/W)

Register 46.13. I2S_TX_PCM2PDM_CONF1_REG (0x0044)

(reserved)							(reserved)							(reserved)				(reserved)				I2S_TX_PDM_FP											
3126							3126							2523		2220		90															
0000000							0000000							7		7		960												Reset			

I2S_TX_PDM_FS Configures I2S PDM TX upsampling parameter. (R/W)

Register 46.14. I2S_TX_TDM_CTRL_REG (0x0054)

[illegible]

I2S_TX_TDM_CHAN n _EN (n : 0-15) Configures whether to enable the valid data output of I2S TX TDM channel n .

0: Channel TX data is controlled by [I2S_TX_CHAN_EQUAL](#) and [I2S_SINGLE_DATA](#). See Section [46.9.2.1](#)

1: Enable

(R/W)

I2S_TX_TDM_TOT_CHAN_NUM Configures the total number of channels in use in I2S TX TDM mode.
Total channel number in use = I2S_TX_TDM_TOT_CHAN_NUM + 1. (R/W)

I2S_TX_TDM_SKIP_MSK_EN Configures the data to be sent in GDMA TX buffer.

0: Data stored in GDMA TX buffer is used by enabled channels and will not be read by channels that are not enabled.

1: Data stored in GDMA TX buffer is read by all channels and will be skipped by channels that are not enabled.

(R/W)

Register 46.15. I2S_RX_TIMING_REG (0x0058)

(reserved)		I2S_RX_BCK_IN_DM		(reserved)		I2S_RX_WS_IN_DM		(reserved)		I2S_RX_BCK_OUT_DM		(reserved)		I2S_RX_WS_OUT_DM		(reserved)		I2S_RX_SD3_IN_DM		(reserved)		I2S_RX_SD2_IN_DM		(reserved)		I2S_RX_SD1_IN_DM		(reserved)		I2S_RX_SD_IN_DM	
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0x0	0	0	0x0	0	0	0x0	0	0	0x0	0	0	0x0	0	0	0x0	0	0	0x0	0	0	0x0	0	0	0x0	0	0	0x0	0	0x0

Reset

I2S_RX_SD_IN_DM Configures the delay mode of I2S RX SD input signal.

0: Bypass

1: Delay by rising edge

2: Delay by falling edge

3: Reserved

(R/W)

I2S_RX_SD1_IN_DM (for I2S0 only) Configures the delay mode of I2S RX SD1 input signal. For detailed configuration values, please refer to [I2S_RX_SD_IN_DM](#). (R/W)

I2S_RX_SD2_IN_DM (for I2S0 only) Configures the delay mode of I2S RX SD2 input signal. For detailed configuration values, please refer to [I2S_RX_SD_IN_DM](#). (R/W)

I2S_RX_SD3_IN_DM (for I2S0 only) Configures the delay mode of I2S RX SD3 input signal. For detailed configuration values, please refer to [I2S_RX_SD_IN_DM](#). (R/W)

I2S_RX_WS_OUT_DM Configures the delay mode of I2S RX WS output signal. For detailed configuration values, please refer to [I2S_RX_SD_IN_DM](#). (R/W)

I2S_RX_BCK_OUT_DM Configures the delay mode of I2S RX BCK output signal. For detailed configuration values, please refer to [I2S_RX_SD_IN_DM](#). (R/W)

I2S_RX_WS_IN_DM Configures the delay mode of I2S RX WS input signal. For detailed configuration values, please refer to [I2S_RX_SD_IN_DM](#). (R/W)

I2S_RX_BCK_IN_DM Configures the delay mode of I2S RX BCK input signal. For detailed configuration values, please refer to [I2S_RX_SD_IN_DM](#). (R/W)

Register 46.16. I2S_TX_TIMING_REG (0x005C)

(reserved)				I2S_TX_BCK_IN_DM				(reserved)				I2S_TX_WS_IN_DM				(reserved)				I2S_TX_BCK_OUT_DM				(reserved)				I2S_TX_WS_OUT_DM				(reserved)										I2S_TX_SD1_OUT_DM				(reserved)				I2S_TX_SD_OUT_DM			
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15													6	5	4	3	2	1	0																		
0	0	0x0	0	0	0x0	0	0	0x0	0	0	0x0	0	0	0x0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0x0	0	0	0	0	0x0																		

Reset

I2S_TX_SD_OUT_DM Configures the delay mode of I2S TX SD output signal.

- 0: Bypass
 - 1: Delay by rising edge
 - 2: Delay by falling edge
 - 3: Reserved
- (R/W)

I2S_TX_SD1_OUT_DM (for I2S0 only) Configures the delay mode of I2S TX SD1 output signal. For detailed configuration values, please refer to [I2S_TX_SD_OUT_DM](#). (R/W)

I2S_TX_WS_OUT_DM Configures the delay mode of I2S TX WS output signal. For detailed configuration values, please refer to [I2S_TX_SD_OUT_DM](#). (R/W)

I2S_TX_BCK_OUT_DM Configures the delay mode of I2S TX BCK output signal. For detailed configuration values, please refer to [I2S_TX_SD_OUT_DM](#). (R/W)

I2S_TX_WS_IN_DM Configures the delay mode of I2S TX WS input signal. For detailed configuration values, please refer to [I2S_TX_SD_OUT_DM](#). (R/W)

I2S_TX_BCK_IN_DM Configures the delay mode of I2S TX BCK input signal. For detailed configuration values, please refer to [I2S_TX_SD_OUT_DM](#). (R/W)

Register 46.17. I2S_LC_HUNG_CONF_REG (0x0060)

(reserved)																				I2S_LC_FIFO_TIMEOUT_ENA				I2S_LC_FIFO_TIMEOUT_SHIFT				I2S_LC_FIFO_TIMEOUT			
31																				12	11	10	8	7					0		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0x10				Reset					

I2S_LC_FIFO_TIMEOUT Configures FIFO timeout threshold. The FIFO hung counter is incremented by one each time I2S is in an error state and the tick counter reaches its threshold. [I2S_TX_HUNG_INT](#) or [I2S_RX_HUNG_INT](#) interrupt will be triggered when FIFO hung counter is equal to the FIFO timeout threshold. (R/W)

I2S_LC_FIFO_TIMEOUT_SHIFT Configures tick counter threshold. The tick counter is incremented by one each time I2S is in an error state and it is on the rising edge in each system clock (APB). The tick counter is reset when counter value $\geq 88000/2^{I2S_LC_FIFO_TIMEOUT_SHIFT}$. (R/W)

I2S_LC_FIFO_TIMEOUT_ENA Configures whether to enable FIFO timeout.

0: Disable

1: Enable

(R/W)

Register 46.18. I2S_CONF_SIGLE_DATA_REG (0x0068)

I2S_SINGLE_DATA																																
31																															0	
																															0	Reset

I2S_SINGLE_DATA Configures constant channel data to be sent out. (R/W)

Register 46.19. I2S_STATE_REG (0x006C)

(reserved)																															I2S_TX_IDLE			
31																															1	0		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	Reset

I2S_TX_IDLE Represents the TX unit state.

0: I2S TX unit is working

1: I2S TX unit is in idle state

(RO)

Register 46.20. I2S_ETM_CONF_REG (0x0070)

(reserved)												I2S_ETM_RX_RECEIVE_WORD_NUM												I2S_ETM_TX_SEND_WORD_NUM																																																																																																											
31												20												19												10												9												0																																																																							
0												0												0												0												0												0												0												0												0x40												0x40												Reset											

I2S_ETM_TX_SEND_WORD_NUM Configures the threshold of triggering ETM [I2S_TX_X_WORDS_SENT](#) event. When transmitting word number of I2S_ETM_TX_SEND_WORD_NUM [9:0], I2S will trigger the corresponding ETM event. (R/W)

I2S_ETM_RX_RECEIVE_WORD_NUM Configures the threshold of triggering ETM [I2S_RX_X_WORDS_RECEIVED](#) event. When receiving word number of I2S_ETM_RX_RECEIVE_WORD_NUM [9:0], I2S will trigger the corresponding ETM event. (R/W)

Register 46.21. I2S_FIFO_CNT_REG (0x0074)

31	30	0
0	0x000000	

Reset

I2S_TX_FIFO_CNT Configures the TX FIFO counter value. (RO)

I2S_TX_FIFO_CNT_RST Configure whether to reset the TX FIFO counter.

0: No effect

1: Reset

(WT)

Register 46.22. I2S_BCK_CNT_REG (0x0078)

Diagram illustrating the I2S_TX_BCK_CNT register structure. The register is 32 bits wide. Bit 31 is labeled I2S_TX_BCK_CNT_RST. Bits 30 to 0 are labeled I2S_TX_BCK_CNT. The register value is shown as 0x000000.

I2S_TX_BCK_CNT Configures the TX BCK counter value. (RO)

I2S_TX_BCK_CNT_RST Configures whether to reset TC BCK counter.

0: No effect

1: Reset

(WT)

Register 46.23. I2S_CLK_GATE_REG (0x007C)

[illegible]

I2S_CLK_EN Configures whether to enable clock gate.

0: Disable

1: Enable

(R/W)

Register 46.24. I2S_DATE_REG (0x0080)

(reserved)				I2S_DATE																								
31		28		27																								0
0	0	0	0		0x2303240																							Reset

I2S_DATE Version control register. (R/W)